

**МИНОБРНАУКИ РОССИИ**  
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**  
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**  
**«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)**  
**Кафедра МО ЭВМ**

**ОТЧЕТ**  
**по лабораторной работе №4**  
**по дисциплине «Параллельные алгоритмы»**  
**Тема: Алгоритмы GPGPU**

Студент гр. 3343

\_\_\_\_\_

Пименов П.В.

Преподаватель

\_\_\_\_\_

Сергеева Е.И.

Санкт-Петербург

2025

### **Задание.**

Реализовать алгоритм для исполнения на GPU. Реализацию выполнить на OpenCL/CUDA (выбор фреймворка исходя из доступного аппаратного обеспечения).

1. Блочное умножение матриц. Выполнить тестирование и анализ производительности. Оценить ускорение по сравнению с лр1.

2. Сортировка (алгоритм на выбор из класса "быстрых" сортировок, например merge sort). Выполнить тестирование (сравнение с cpi реализацией) и анализ производительности.

## Выполнение работы.

Оба задания успешно выполнены. Использован фреймворк CUDA (поскольку на тестовом компьютере видеокарта NVIDIA). В качестве алгоритма сортировки выбрана Быстрая сортировка (Qsort).

### 1. Детали реализации

#### 1.1. Матрицы

- Ядро `matMulTiledKernel` реализовано как `tiled kernel` с использованием `shared memory`:
  - $TILE = 16$ , блоки  $(TILE, TILE)$ , сетка  $grid = \lceil N / TILE \rceil \times \lceil N / TILE \rceil$ .
  - Тайлы матриц  $A$  и  $B$  загружаются в `shared memory` ( $A_s$  и  $B_s$ ); затем выполняется внутренний цикл по  $TILE$  и аккумулируется сумма; предусмотрена обработка краёв (`out-of-bounds`) с подстановкой  $0$ .
  - Типы данных: `double` (двойная точность) на хосте и устройстве; байтовые подсчёты используют `sizeof(double)`.
  - Замер времени: для каждой итерации используется `cudaEvent` (`start/stop`); измеряется суммарное время  $H2D$  (копирование  $A$  и  $B$ ) + `kernel` +  $D2H$  (копирование  $C$ ). В конце усредняется по `mat_iters`.

#### 1.2. Сортировка

- Для сортировки использована библиотека `Thrust`:
  - На каждой итерации создаётся `thrust::device_vector<int> dvec = a` (копирование  $H2D$ ).
  - Выполняется `thrust::sort(dvec.begin(), dvec.end());` (внутри реализован подходящий алгоритм, для `int` часто это `radix sort` под капотом).
  - После сортировки данные копируются обратно `thrust::copy(dvec.begin(), dvec.end(), a.begin());` ( $D2H$ ).
  - Для синхронизации вызывается `cudaDeviceSynchronize()`, а измерение времени делается средствами `std::chrono` вокруг всей последовательности  $H2D + \text{sort} + D2H$  (`wall-clock`).

- CPU-реализация сортировки — рекурсивная быстрая сортировка quicksort\_cpu (in-place), измеряется через std::chrono (wall-clock).

### 1.3. Замечания по реализации

- GPU-замеры включают накладные расходы копирования данных на устройство и обратно; это важно учитывать при интерпретации speedup.

- Для многократных итераций в main.cpp память GPU выделяется и освобождается внутри цикла применения итераций (в текущем варианте возможны дополнительные накладные расходы на cudaMalloc/cudaFree, если они выполняются в цикле), что влияет на итоговые времена; в представленных файлах замеры добавлялись в диапазоне mat\_iters и усреднялись.

## 2. Методика измерений и обработка данных

### 2.1. Единицы и CSV

- results\_lb1\_patched.csv: size, threads, method, time\_s (секунды) — CPU block (threads=16).

- results\_matmul.csv: size, iterations, gpu\_time\_ms (миллисекунды) — GPU (H2D+kernel+D2H).

- results\_sort.csv: size, iterations, cpu\_time\_ms, gpu\_time\_ms, speedup — сортировка.

### 2.2. Вычисление speedup

Матрицы:

- Из lb1: блоковое время block\_time\_s (для threads==16) используется как опорное.

- Из matmul: gpu\_time\_ms переводится в секунды  $gpu\_sec = gpu\_time\_ms / 1000$ .

-  $Speedup = block\_time\_s / gpu\_sec$ .

Сортировка:

-  $Speedup = cpu\_time\_ms / gpu\_time\_ms$  (оба в ms)

### 2.3. Обоснование подхода

- Сравнение end-to-end (H2D+kernel+D2H) отражает реальную стоимость использования GPU для полного сценария.

- Дополнительные метрики (kernel only, H2D отдельно) в текущем наборе измерений не сохранены; для детального анализа рекомендуется дополнительно отдельно измерять kernel execution time (cudaEvent вокруг kernel) и отдельно суммарное время копирования.

### 3. Графическое представление результатов тестирования

Результаты тестирования представлены на Рисунках 1 и 2. На Рисунке 1 в качестве опоры сравнения выбрана реализация блочного умножения матриц, запущенная на cpi на 16 потоках. На Рисунке 2 – опорная линия – обычная cpi-реализация qsort.

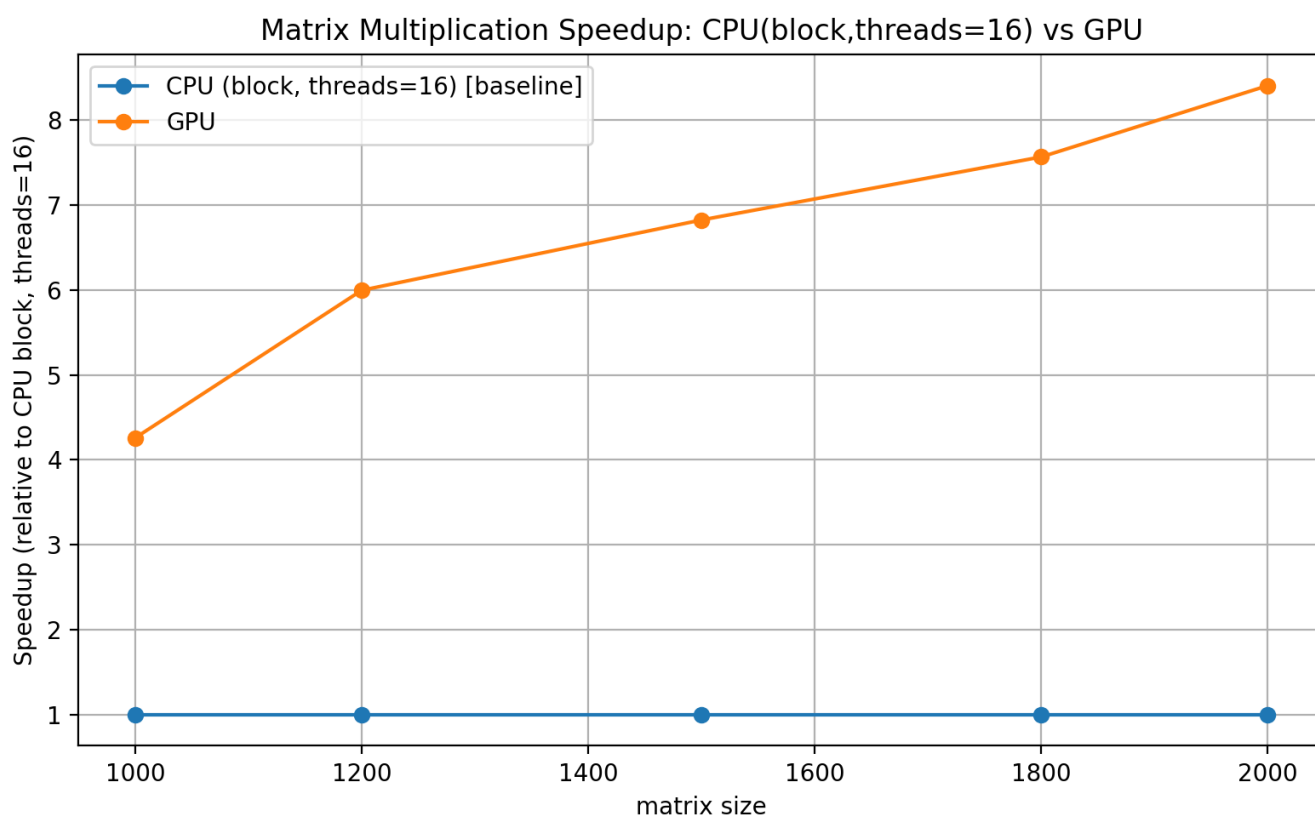


Рисунок 1 – Сравнение ускорения Matrix Block CPU vs GPU

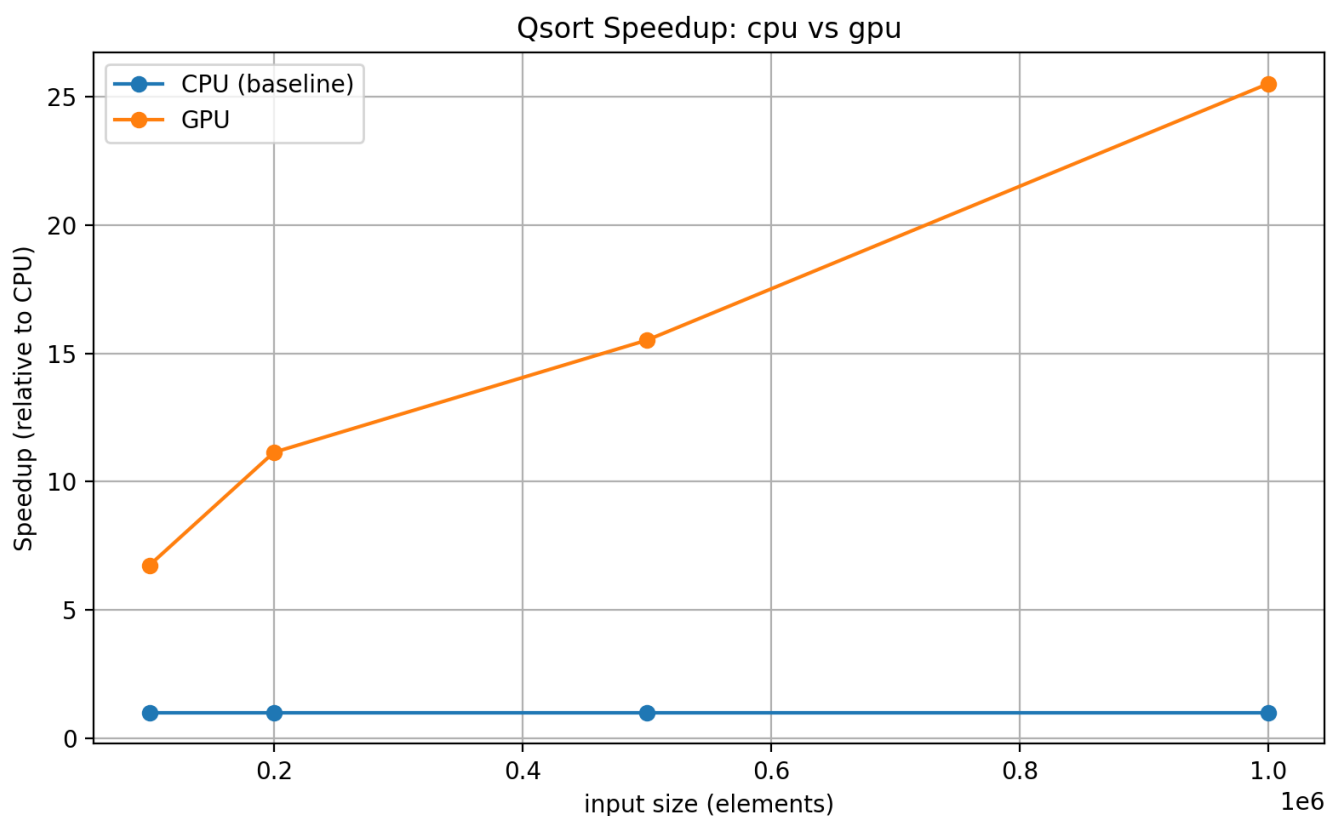


Рисунок 2 – Сравнение ускорения Qsort CPU vs GPU

#### 4. Численные результаты тестирования

Все цифры приведены дословно, единицы измерений указаны в названиях столбцов.

Таблица 4.1. Matrix Block CPU vs GPU

| size | cpu_time_s | gpu_time_s | speedup |
|------|------------|------------|---------|
| 1000 | 0.126551   | 0.0089432  | 14.15   |
| 1200 | 0.216430   | 0.0100506  | 21.54   |
| 1500 | 0.400386   | 0.0186470  | 21.47   |
| 1800 | 0.744511   | 0.0314277  | 23.71   |
| 2000 | 1.109640   | 0.0418271  | 26.53   |

Таблица 4.2. Qsort CPU vs GPU

| size   | cpu_time_ms | gpu_time_ms | speedup |
|--------|-------------|-------------|---------|
| 100000 | 5.2311      | 0.960348    | 5.45    |
| 200000 | 10.7632     | 1.00628     | 10.70   |
| 500000 | 28.2283     | 2.2815      | 12.37   |

|         |         |         |       |
|---------|---------|---------|-------|
| 1000000 | 59.1488 | 2.76991 | 21.35 |
|---------|---------|---------|-------|

## 5. Анализ полученных данных

На основании полученных данных приводятся фактологические наблюдения.

- Для умножения матриц GPU даёт значительное ускорение по сравнению с многопоточной CPU-блоковой реализацией с 16 потоками: в диапазоне измеренных размеров наблюдаются ускорения  $\approx 14 - 26.5$ .

- Для сортировки (Thrust) наблюдается рост ускорения с увеличением размера входа: от  $\approx 5.5\times$  при 100k до  $\approx 21.4\times$  при 1M. Измерения включают накладные расходы копирования данных на GPU и обратно.

- GPU-тайминги в main.cpp измерялись с помощью cudaEvent (включая H2D и D2H). Поскольку CSV содержит полные времена H2D+kernel+D2H, сравнение отражает реальную пользовательскую стоимость использования GPU для полных операций, а не только kernel execution time.

## Выводы.

Все задания лабораторной выполнены успешно. По представленным измерениям можно сделать вывод, что GPU даёт многократное ускорение как для матричного умножения, так и для сортировки.

## ПРИЛОЖЕНИЕ А

### Файл main.cpp:

```
#include <algorithm>
#include <cassert>
#include <chrono>
#include <cstring>
#include <fstream>
#include <iostream>
#include <random>
#include <vector>

// CUDA + Thrust
#include <cuda_runtime.h>
#include <thrust/device_vector.h>
#include <thrust/sort.h>

#define TILE 16

// CUDA error check
#define CUDA_CHECK(call) \
do { \
    cudaError_t err = call; \
    if (err != cudaSuccess) { \
        std::cerr << "CUDA error at " << __FILE__ << ":" << __LINE__ \
        << " code=" << err << " \"" << cudaGetErrorString(err) \
        << "\"\n"; \
        std::exit(EXIT_FAILURE); \
    } \
} while (0)

// GPU kernel: tiled matrix multiplication (A * B = C), dimension NxN
__global__ void matMulTiledKernel(const double* __restrict__ A,
                                const double* __restrict__ B,
                                double* __restrict__ C, int N) {
    __shared__ double As[TILE][TILE];
    __shared__ double Bs[TILE][TILE];

    int bx = blockIdx.x, by = blockIdx.y;
    int tx = threadIdx.x, ty = threadIdx.y;

    int row = by * TILE + ty;
    int col = bx * TILE + tx;

    double sum = 0.0;
    int numTiles = (N + TILE - 1) / TILE;
    for (int m = 0; m < numTiles; ++m) {
        int aCol = m * TILE + tx;
        int bRow = m * TILE + ty;
        As[ty][tx] = (row < N && aCol < N) ? A[row * N + aCol] : 0.0;
        Bs[ty][tx] = (bRow < N && col < N) ? B[bRow * N + col] : 0.0;
        __syncthreads();

        for (int k = 0; k < TILE; ++k) {
            sum += As[ty][k] * Bs[k][tx];
        }
        __syncthreads();
    }
    if (row < N && col < N) {
        C[row * N + col] = sum;
    }
}
```



```

// Simple QuickSort for CPU (in-place)
void quicksort_cpu(std::vector<int>& a, int lo, int hi) {
    if (lo >= hi)
        return;
    int i = lo, j = hi;
    int pivot = a[lo + (hi - lo) / 2];
    while (i <= j) {
        while (a[i] < pivot)
            ++i;
        while (a[j] > pivot)
            --j;
        if (i <= j) {
            std::swap(a[i], a[j]);
            ++i;
            --j;
        }
    }
    if (lo < j)
        quicksort_cpu(a, lo, j);
    if (i < hi)
        quicksort_cpu(a, i, hi);
}

int main(int argc, char** argv) {
    // Configurable sizes
    std::vector<int> mat_sizes = {
        1000, 1200, 1500, 1800,
        2000}; // можно расширять, следите за памятью GPU
    std::vector<int> sort_sizes = {100000, 200000, 500000, 1000000};

    int mat_iters = 3; // повторов для усреднения
    int sort_iters = 5;

    std::cout << "Starting tests (CUDA). Make sure you run on machine with "
                "NVIDIA GPU.\n";

    // Output CSV files
    // Now we only record GPU timings for matmul (CPU naive removed)
    std::ofstream fmat("results_matmul.csv");
    fmat << "size,iterations,gpu_time_ms\n";

    // Matmul tests (GPU only), using double precision matrices
    for (int N : mat_sizes) {
        std::cout << "Testing matmul N=" << N << "... \n";
        size_t elements = (size_t)N * N;
        size_t bytes = elements * sizeof(double);
        // Host allocate (double precision)
        std::vector<double> A(elements), B(elements), C_gpu(elements);

        std::mt19937 rng(12345);
        std::uniform_real_distribution<double> dist(0.0, 1.0);
        for (size_t i = 0; i < elements; ++i) {
            A[i] = dist(rng);
            B[i] = dist(rng);
        }

        // GPU time including H2D, kernel, D2H
        double gpu_total_ms = 0.0;
        bool gpu_ok = true;
        try {
            double *dA = nullptr, *dB = nullptr, *dC = nullptr;
            CUDA_CHECK(cudaMalloc((void**)&dA, bytes));
            CUDA_CHECK(cudaMalloc((void**)&dB, bytes));

```

```

    CUDA_CHECK(cudaMalloc((void**)&dC, bytes));

    // prepare events for timing
    cudaEvent_t start, stop;
    CUDA_CHECK(cudaEventCreate(&start));
    CUDA_CHECK(cudaEventCreate(&stop));

    dim3 block(TILE, TILE);
    dim3 grid((N + TILE - 1) / TILE, (N + TILE - 1) / TILE);

    for (int it = 0; it < mat_iters; ++it) {
        CUDA_CHECK(cudaEventRecord(start));
        CUDA_CHECK(
            cudaMemcpy(dA, A.data(), bytes, cudaMemcpyHostToDevice));
        CUDA_CHECK(
            cudaMemcpy(dB, B.data(), bytes, cudaMemcpyHostToDevice));

        matMultTiledKernel<<<grid, block>>>(dA, dB, dC, N);
        CUDA_CHECK(cudaGetLastError());

        CUDA_CHECK(cudaMemcpy(C_gpu.data(), dC, bytes,
                               cudaMemcpyDeviceToHost));
        CUDA_CHECK(cudaEventRecord(stop));
        CUDA_CHECK(cudaEventSynchronize(stop));
        float ms = 0.0f;
        CUDA_CHECK(cudaEventElapsedTime(&ms, start, stop));
        gpu_total_ms += ms;
    }

    CUDA_CHECK(cudaFree(dA));
    CUDA_CHECK(cudaFree(dB));
    CUDA_CHECK(cudaFree(dC));
    CUDA_CHECK(cudaEventDestroy(start));
    CUDA_CHECK(cudaEventDestroy(stop));
} catch (...) {
    gpu_ok = false;
    std::cerr << "GPU run failed (out of memory or other CUDA error). "
                "Skipping GPU for this size.\n";
}

if (gpu_ok) {
    double gpu_avg = gpu_total_ms / mat_iters;
    std::cout << "N=" << N << " gpu_avg_ms=" << gpu_avg << "\n";
    fmat << N << "," << mat_iters << "," << gpu_avg << "\n";
} else {
    // write an empty gpu_time entry if GPU failed for this size
    fmat << N << "," << mat_iters << ",\n";
}
}
fmat.close();

// Sorting tests (unchanged)
std::ofstream fsort("results_sort.csv");
fsort << "size,iterations,cpu_time_ms,gpu_time_ms,speedup\n";

for (int N : sort_sizes) {
    std::cout << "Testing sort N=" << N << "... \n";
    std::mt19937 rng(1234567);
    std::uniform_int_distribution<int> dist(0, 1000000000);

    double cpu_total_ms = 0.0;
    for (int it = 0; it < sort_iters; ++it) {
        std::vector<int> a(N);

```

```

        for (int i = 0; i < N; ++i)
            a[i] = dist(rng);
        auto t0 = std::chrono::high_resolution_clock::now();
        quicksort_cpu(a, 0, N - 1);
        auto t1 = std::chrono::high_resolution_clock::now();
        cpu_total_ms +=
            std::chrono::duration<double, std::milli>(t1 - t0).count();
    }
    double cpu_avg = cpu_total_ms / sort_iters;

    // GPU sort with Thrust (including host->device and device->host)
    double gpu_total_ms = 0.0;
    for (int it = 0; it < sort_iters; ++it) {
        std::vector<int> a(N);
        for (int i = 0; i < N; ++i)
            a[i] = dist(rng);
        // H2D + sort + D2H
        auto t0 = std::chrono::high_resolution_clock::now();
        thrust::device_vector<int> dvec = a; // copies host->device
        thrust::sort(dvec.begin(), dvec.end());
        thrust::copy(dvec.begin(), dvec.end(), a.begin()); // device->host
        CUDA_CHECK(cudaDeviceSynchronize());
        auto t1 = std::chrono::high_resolution_clock::now();
        gpu_total_ms +=
            std::chrono::duration<double, std::milli>(t1 - t0).count();
    }
    double gpu_avg = gpu_total_ms / sort_iters;
    double speedup = cpu_avg / gpu_avg;
    std::cout << "sort N=" << N << " cpu_avg_ms=" << cpu_avg
        << " gpu_avg_ms=" << gpu_avg << "\n";
    fsort << N << "," << sort_iters << "," << cpu_avg << "," << gpu_avg
        << "," << speedup << "\n";
}
fsort.close();

std::cout
    << "Done. Results written to results_matmul.csv and results_sort.csv\n";

return 0;
}

```

### results\_lb1.csv:

```

size,threads,method,time_s
1000,16,Threads,0.126551
1200,16,Threads,0.21643
1500,16,Threads,0.400386
1800,16,Threads,0.744511
2000,16,Threads,1.10964

```

### results\_matmul.csv:

```

size,iterations,gpu_time_ms
1000,3,29.7279
1200,3,36.0956
1500,3,58.6658
1800,3,98.3818
2000,3,131.972

```

### results\_qsort.csv:

```

size,iterations,cpu_time_ms,gpu_time_ms,speedup
100000,5,5.152,0.764213,6.74157
200000,5,11.3741,1.02082,11.1421

```

500000,5,30.7537,1.98178,15.5182  
1000000,5,63.8155,2.49988,25.5274